Lecture L24

*Blind review — please score this lecture on row **L24** in the Excel form (1–10 per dimension).*

Normalization Forms and Database Schema Design (1NF–BCNF)

Learning Objectives

By the end of this lecture, you will be able to:

1. **Remember** the definitions of 1NF, 2NF, 3NF, and BCNF, and state what each form prohibits.
2. **Understand** why partial and transitive dependencies cause update anomalies, and explain the difference between a candidate key and a superkey.
3. **Apply** the step-by-step normalization process to decompose a given unnormalized relation into 1NF, 2NF, 3NF, and BCNF, identifying all functional dependencies and candidate keys along the way.
4. **Analyze** a given relational schema to determine its current normal form and diagnose which specific functional dependency causes the violation.
5. **Evaluate** trade-offs between full normalization and denormalization, and explain the role of lossless-join and dependency-preservation properties in schema decomposition.

Prerequisites — What We Assume You Already Know

Before we begin, we assume you are comfortable with:

- The basic structure of a relational table: attributes (columns), tuples (rows), and domains (data types).
- The concept of a **primary key** and the uniqueness constraint it enforces.
- The idea of a **functional dependency** (FD), written as $X \to Y$, meaning: if two tuples agree on the attributes in X, they must also agree on the attributes in Y.
- The terms **superkey** (any set of attributes that uniquely identifies a tuple) and **candidate key** (a minimal superkey — no attribute can be removed without losing the uniqueness property).

If any of these feel rusty, take a moment to review them. We will build directly on these ideas.

---

1. Why Normalize? The Problem We Are Solving

1.0 Opening Hook: The Database That Forgets

Imagine you are building a database for a small company that tracks employees, the projects they work on, and the locations of those projects. A naive first attempt might put everything into one big table. It works — until it doesn't.

Here is the unsettling reality: your single-table design will silently corrupt data through ordinary operations — renaming a project, hiring an employee before assigning them, or deleting a departing employee. Each of these is a completely routine business event, yet each will leave your database in an inconsistent

state that no error message will catch. The table *looks* correct, but it now contains lies: duplicate names, missing projects, or phantom relationships.

This is not a bug in your application code. It is a structural defect in your schema. And the only way to fix it permanently is to understand exactly *why* the structure permits these anomalies — which is what normalization reveals.

1.1 A Single-Table Design and Its Anomalies

Suppose we store all information in one table called `EMPLOYEE_PROJECTS`:

```
EMP_ID | EmpName   | ProjID | ProjName | Location | Hours
-------|-----------|--------|----------|----------|------
E01    | Smith     | P1     | Alpha    | NYC      | 20
E01    | Smith     | P1     | Alpha    | BOS      | 20
E01    | Smith     | P2     | Beta     | CHI      | 15
E02    | Jones     | P3     | Gamma    | SFO      | 30
```

At first glance this looks like a normal table. However, three specific anomalies arise from this design:

- **Update anomaly:** If the project named "Alpha" gets renamed to "AlphaPlus," we must update *every single row* that contains `ProjName = 'Alpha'`. If we miss one, we have inconsistent data — two rows now silently disagree on the name of project P1.
- **Insertion anomaly:** We want to add a new project `P4` with name `Delta` and no employees assigned yet. We cannot insert it because there is no `EMP_ID` value to complete the row — `EMP_ID` is part of the key and cannot be null. The database forces us to invent a fake employee just to record a project's existence.
- **Deletion anomaly:** If employee `E02` leaves and we delete their only row, we lose all information about project `P3` and its name `Gamma` entirely, even though the project still exists. Deleting a person deletes a project — a fact we never intended to couple.

These anomalies are not merely theoretical. They cause data corruption, wasted storage, and application bugs. Normalization is the systematic technique that eliminates these anomalies by *decomposing* one badly designed table into several well-designed tables.

1.2 The Core Idea: One Fact in One Place

The guiding principle of normalization is:

> Every attribute should describe the key, the whole key, and nothing but the key.

This is a mnemonic, but it encodes a precise technical condition. When we violate this principle — for example, by storing an employee's name in a table where the key is a combination of employee, project, and location — the employee name depends on only part of the key (EMP_ID alone). So it gets repeated unnecessarily, and every repetition is an opportunity for inconsistency — an update anomaly waiting to happen. Normalization identifies these violations and splits the table so that each fact (employee name, project name, hours worked) lives in exactly one place.

Try this yourself: Before reading further, look at the table above and ask: which attributes depend on the whole key {EMP_ID, ProjID, Location}, and which depend on only part of it? Your answer will anticipate exactly where we decompose next.

1.3 The Two Non-Negotiable Properties of Decomposition

When we split a table, we must ensure two properties remain true:

1. **Lossless-join property:** We must be able to reconstruct the original table exactly by joining the decomposed tables. No extra, "spurious" tuples should appear. Think of this as the database equivalent of reversing a chemical reaction — you must recover the original substance, not a different mixture.
2. **Dependency preservation property:** All original functional dependencies should be enforceable directly on the decomposed tables, without needing to join them together to check a constraint. This matters because joins are expensive, and a constraint that requires a join on every insert is a performance disaster waiting to happen.

We will verify these properties as we proceed. For now, remember: a decomposition that loses information or loses constraints is not a valid normalization.

---

2. First Normal Form (1NF) — The Foundation

2.1 Intuition Before Formalism

The relational model is built on the idea of a flat, two-dimensional grid. At every row-column intersection, we expect exactly one atomic value — a single number, a single string, a single date. If a column contains a list, a set, or a nested table inside it, then we cannot uniformly use relational operators like `SELECT`, `JOIN`, or `GROUP BY` on that column. 1NF is the rule that says: "Flatten everything. No lists inside cells."

Analogy: Think of a spreadsheet. Every cell holds one value. If you put a whole mini-spreadsheet inside a cell, you can no longer sort, filter, or formula-reference that inner data uniformly with the outer data. 1NF forces every cell to be a single data point, so the entire table is a uniform grid that all relational operators can traverse.

2.2 Formal Definition

A relation schema is in **First Normal Form (1NF)** if the domain of every attribute includes only atomic (indivisible) values, and every tuple contains exactly one value from that domain for each attribute. In other words, 1NF disallows:

- Multivalued attributes (e.g., a column `PhoneNumbers` storing a set of numbers).
- Composite attributes (e.g., a column `Address` that itself is composed of `Street`, `City`, `Zip`).
- Nested relations (a table inside a table).

2.3 Worked Example: Breaking a Nested Structure

Let us start with a design that is *not* even in 1NF:

```
EMP_ID | EmpName   | Projects(ProjID, ProjName, Locations(*))
-------|-----------|-----------------------------------------
E01    | Smith     | { (P1, Alpha, {NYC, BOS}), (P2, Beta, {CHI}) }
E02    | Jones     | { (P3, Gamma, {SFO}) }
```

Here, `Projects` is a nested relation — a table inside a cell. Inside it, `Locations` is a multivalued set. This is a clear 1NF violation.

Step 1: Flatten the nested relation into tuples. We expand each project entry into its own row and each location into its own row. The result is a flat table:

```
EMP_ID | EmpName | ProjID | ProjName | Location
-------|---------|--------|----------|---------
```

```
E01    | Smith   | P1     | Alpha    | NYC
E01    | Smith   | P1     | Alpha    | BOS
E01    | Smith   | P2     | Beta     | CHI
E02    | Jones   | P3     | Gamma    | SFO
```

Step 2: Identify the new primary key. In the original design, `EMP_ID` alone might have been the key. Now, because an employee can have multiple projects and a project can have multiple locations, a single `EMP_ID` no longer uniquely identifies a row. We need the combination of all three: `{EMP_ID, ProjID, Location}`. No subset of this combination guarantees uniqueness anymore.

Check: Is every cell atomic? Yes — each contains a single string or number. The table is now in 1NF.

2.4 Misconception Alert

A common misconception is: "If I split `Address` into `Street`, `City`, `Zip`, I am normalizing to 1NF." That is true only if `Address` was a composite attribute. Splitting an already-atomic attribute into smaller pieces is not 1NF normalization — it is a design choice about granularity. 1NF is about removing non-atomic domains, not about arbitrarily breaking strings apart.

2.5 Quick Check — Test Yourself

Is the following table in 1NF?

```
StudentID | Name   | Courses
----------|--------|--------------------------
S01       | Alice  | Math, Physics, Chemistry
S02       | Bob    | Biology
```

Answer: No. The `Courses` column contains a comma-separated list — a multivalued attribute. To reach 1NF, each course must appear in its own row.

---

3. Second Normal Form (2NF) — Eliminating Partial Dependencies

3.1 The Problem 2NF Solves

Consider again our 1NF table from Section 2.3. It is flat and atomic — so it satisfies 1NF. But the anomalies from Section 1.1 are still present. Why? Because 1NF only guaranteed atomic cells; it said nothing about which attributes depend on which parts of the key.

Here is the tension: the table is in 1NF, yet it still corrupts data on updates, blocks inserts, and destroys information on deletes. This is the intellectual tension that motivates 2NF — 1NF was necessary but not sufficient. We need a stronger condition that examines the functional dependencies *inside* the key.

Analogy: 1NF is like ensuring every box in a warehouse holds exactly one item. But if you store the item's label inside a box that also holds a shipping address, and you have multiple boxes for the same item with different addresses, you still have to update the label in every box when it changes. 2NF says: the label belongs with the item, not with the item-and-address combination.

3.2 Formal Definition

A relation schema is in **Second Normal Form (2NF)** if:

1. It is in 1NF, and

2. Every non-prime attribute (an attribute that is not part of any candidate key) is **fully functionally dependent** on every candidate key — not just on a proper subset of a candidate key.

In simpler terms: no non-key attribute depends on only part of a candidate key. If the key is composite, every non-key attribute must depend on the entire key, not just a piece of it.

3.3 Worked Example: Spotting Partial Dependencies

Let us examine our 1NF table with candidate key `{EMP_ID, ProjID, Location}`:

```
EMP_ID → EmpName          (EmpName depends on EMP_ID alone - partial dependency)
ProjID → ProjName         (ProjName depends on ProjID alone - partial dependency)
{EMP_ID, ProjID} → Hours  (Hours depends on employee and project, but not location)
```

Here is the crucial observation: `EmpName` depends only on `EMP_ID`, which is a proper subset of the candidate key. `ProjName` depends only on `ProjID`, another proper subset. `Hours` depends on `{EMP_ID, ProjID}`, also a proper subset (it does not need `Location`). These are all **partial dependencies** — non-prime attributes that depend on part of a candidate key rather than the whole key. This is exactly what 2NF prohibits.

Decomposition to 2NF: We split the table so that each partial dependency gets its own relation where its determinant is the full key:

- **EMPLOYEE(EMP_ID, EmpName)** — key: `{EMP_ID}`, and `EmpName` now depends on the whole key.
- **PROJECT(ProjID, ProjName)** — key: `{ProjID}`, and `ProjName` depends on the whole key.
- **WORKS_ON(EMP_ID, ProjID, Location, Hours)** — key: `{EMP_ID, ProjID, Location}`. `Hours` still depends on `{EMP_ID, ProjID}`, which is a subset of this key. This is a partial dependency — so this table is *not yet* in 2NF. We will address this in a moment.

Wait — why is `Hours` still a problem? Because in `WORKS_ON`, the candidate key includes `Location`, but `Hours` does not functionally depend on `Location`. It depends only on `{EMP_ID, ProjID}`. So `Hours` is a non-prime attribute that depends on a proper subset of the candidate key — a textbook partial dependency.

Try this yourself: Before reading the fix, ask: what should the key of the hours table really be? If `Hours` depends only on `{EMP_ID, ProjID}`, then `Location` does not belong in the key of that table. The correct 2NF decomposition pulls `Hours` into its own relation with key `{EMP_ID, ProjID}` and leaves `Location` in a separate relation.

Corrected 2NF decomposition:

- **EMPLOYEE(EMP_ID, EmpName)** — key: `{EMP_ID}`
- **PROJECT(ProjID, ProjName)** — key: `{ProjID}`
- **ASSIGNMENT(EMP_ID, ProjID, Hours)** — key: `{EMP_ID, ProjID}`
- **PROJECT_LOCATION(ProjID, Location)** — key: `{ProjID, Location}`

Now every non-prime attribute depends on the full candidate key of its relation. 2NF is satisfied.

3.4 Summary: The 2NF Test

Ask yourself: "Is there a non-key attribute that can be determined by only part of a candidate key?" If yes, the schema violates 2NF. Extract that attribute and its determinant into a new table.

---

4. Third Normal Form (3NF) — Eliminating Transitive Dependencies

4.1 The Problem 3NF Solves

Even after achieving 2NF, a table can still harbor anomalies. Consider a table of employees and their departments:

```
EMP_ID | DeptID | DeptName | DeptLocation
-------|--------|----------|-------------
E01    | D01    | Sales    | NYC
E02    | D01    | Sales    | NYC
E03    | D02    | Eng      | SFO
```

This table is in 2NF. The key is `{EMP_ID}`. There are no partial dependencies because the key is not composite. Yet `DeptName` and `DeptLocation` are repeated for every employee in the same department. If the Sales department moves to Chicago, we must update every employee row — an update anomaly. If we want to create a new department with no employees yet, we cannot insert it — an insertion anomaly. If we delete the last employee in Engineering, we lose the department's name and location — a deletion anomaly.

The tension: 2NF eliminated anomalies caused by partial dependencies on composite keys. But here, the key is simple — a single attribute. The anomalies come from a different mechanism: `DeptName` and `DeptLocation` depend on `DeptID`, which in turn depends on `EMP_ID`. This is a chain of dependencies — a **transitive dependency**. 3NF targets exactly this.

4.2 Formal Definition

A relation schema is in **Third Normal Form (3NF)** if:

1. It is in 2NF, and
2. For every functional dependency $X \to A$ where A is a single non-prime attribute not in X, either:
   - X is a superkey, or
   - A is part of some candidate key (this is the "trivial" escape clause).

Equivalently: no non-prime attribute depends transitively on a candidate key via another non-prime attribute.

Cross-domain connection: This is exactly the same logic as avoiding intermediate variables that introduce inconsistency in a distributed system. If `DeptName` is a function of `DeptID`, and `DeptID` is a function of `EMP_ID`, then `DeptName` is transitively a function of `EMP_ID`. Storing it in the employee table is like caching derived data without an invalidation strategy — every change to the source (`DeptID → DeptName`) requires updating all cached copies. 3NF says: store the source of truth once, and reference it.

4.3 Worked Example: Spotting Transitive Dependencies

In the table above:

- `EMP_ID → DeptID` (given)
- `DeptID → DeptName` (given)
- `DeptID → DeptLocation` (given)
- Therefore: `EMP_ID → DeptID → DeptName` and `EMP_ID → DeptID → DeptLocation` are transitive dependencies.

`DeptName` and `DeptLocation` are non-prime attributes that depend on `DeptID`, which is itself a non-prime attribute (it is not a candidate key — `EMP_ID` is the only candidate key). So the determinant `DeptID` is not a superkey. This violates 3NF.

Decomposition to 3NF:

- **EMPLOYEE(EMP_ID, DeptID)** — key: `{EMP_ID}`; `DeptID` is now a foreign key referencing the department table.
- **DEPARTMENT(DeptID, DeptName, DeptLocation)** — key: `{DeptID}`; `DeptName` and `DeptLocation` now depend directly on the full key, with no transitive chain.

Now, no non-prime attribute depends on another non-prime attribute. The anomalies disappear: renaming a department updates one row; adding an employee-less department inserts into `DEPARTMENT` without touching `EMPLOYEE`; deleting the last employee in a department leaves the department intact.

4.4 Summary: The 3NF Test

Ask yourself: "Is there a non-key attribute that determines another non-key attribute?" If yes, you have a transitive dependency. Extract the dependent attributes and their determinant into a new table where that determinant is the key.

---

5. Boyce-Codd Normal Form (BCNF) — When 3NF Is Not Enough

5.1 The Problem BCNF Solves

3NF handles the vast majority of practical anomalies. But there is a subtle case it misses — when a table has multiple candidate keys that overlap. 3NF allows a dependency to violate the "every determinant must be a superkey" rule if the dependent attribute is part of a candidate key. This escape clause can leave anomalies in place.

The tension: 3NF is a compromise — it relaxes the superkey requirement for certain dependencies to guarantee dependency preservation. But that relaxation has a cost. BCNF asks: what if we remove the escape clause entirely? What anomalies remain when we enforce the stricter rule?

Analogy: Think of 3NF as a building code